Data Analytics Project: IGN Game Reviews Analysis

Dataset Used: ign.csv

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyze game review scores, platform popularity, and genre distribution over time using the `ign.csv` dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (🟢 Easy)

Description:

This scenario focuses on loading the dataset into a Pandas DataFrame and performing initial exploration to understand its structure. It involves removing unnecessary columns and identifying missing values in important fields. Appropriate techniques are applied to handle missing data, ensuring the dataset is clean and consistent. This prepares the data for further analysis and visualization.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
4.	Check for missing values in `score`, `genre`, and `platform`.
5.	Handle missing values: Fill the numeric column `score` with the mean, and the categorical column `genre` with the mode.
6.	Ensure correct data types: Convert `score` → float and `release_year`, `release_month`, `release_day` → integer.

Output : First 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.

```
PS C:\Users\Windows\Desktop\Python Practice Task\Analytics Tasks\Project 2> & C:\Users\Windows\Ap
pData\Local\Python\bin\python.exe "c:/Users/Windows/Desktop/Python Practice Task/Analytics Tasks/
Project 2/Main.py"
First 5 rows:
   Unnamed: 0 score_phrase  ... release_month release_day
0           0      Amazing  ...             9          12
1           1      Amazing  ...             9          12
2           2        Great  ...             9          12
3           3        Great  ...             9          11
4           4        Great  ...             9          11

[5 rows x 11 columns]

Last 5 rows:
       Unnamed: 0 score_phrase  ... release_month release_day
18620       18620         Good  ...             6          29
18621       18621      Amazing  ...             6          29
18622       18622     Mediocre  ...             6          28
18623       18623  Masterpiece  ...             6          28
18624       18624  Masterpiece  ...             6          28

[5 rows x 11 columns]

Shape of dataset:
(18625, 11)
```

Fig 1 : Displaying outputs of head(), Tail(), and shape of dataset.

Conclusion:

The dataset was loaded and basic exploration was performed to understand its structure. Irrelevant columns were removed and missing values were handled effectively. These preprocessing steps improved the overall data quality. The cleaned dataset is now ready for further analysis and visualization.

Scenario 2: Line Graph (Score Trend) + Save (🟢 Easy)

Description:

This scenario focuses on analyzing how game scores change over time by grouping data by release year. The average score for each year is calculated and visualized using a line graph. Proper titles and labels are added for clarity. Finally, the graph is saved as an image file for future use.

Task	Description
1.	Group data by `release_year`.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with `release_year` on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: `plt.savefig("avg_score_trend.png")`.

Graph :

The line graph shows the trend of average game scores over the years. The X-axis represents the release year, while the Y-axis shows the average score. It helps in identifying patterns or changes in scores over time.

Output :

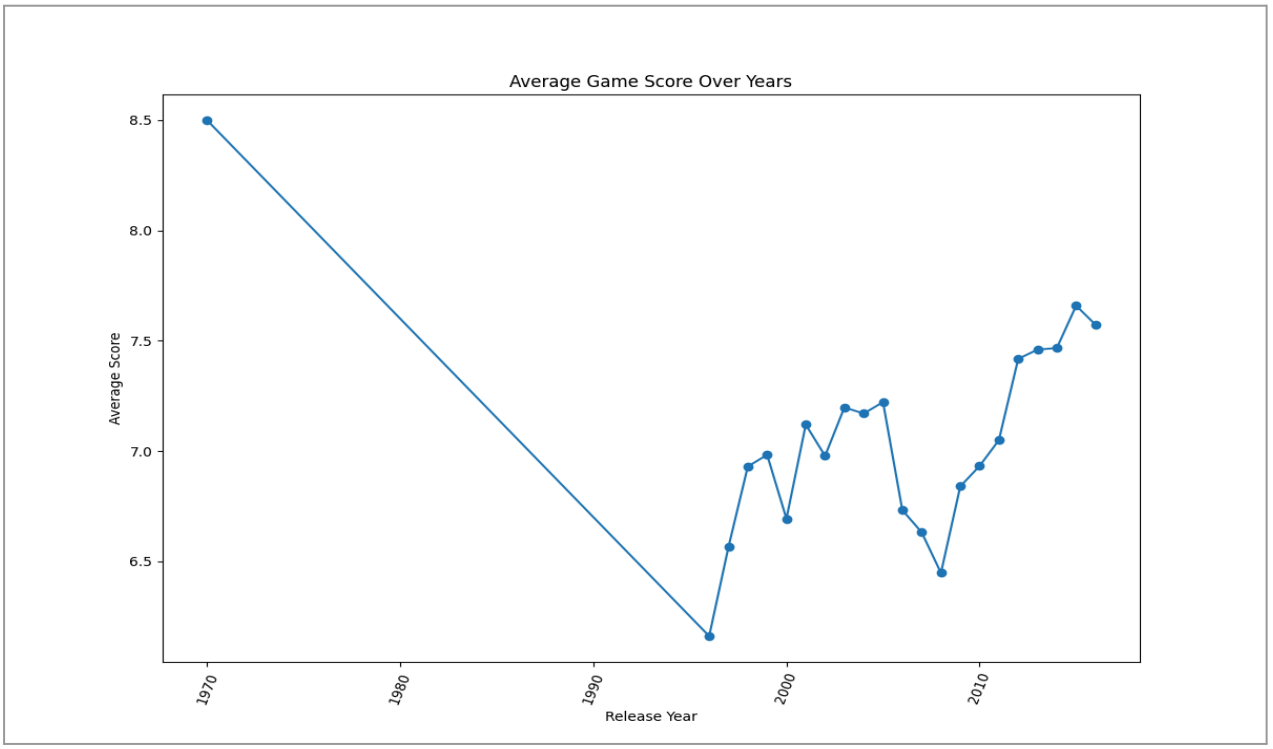

Fig 2 : Average Score Trend

Conclusion:

The line graph effectively visualizes how average game scores change over time. It helps in identifying trends, patterns, and fluctuations across different years. Proper labeling makes the graph easy to understand and interpret. Saving the graph ensures it can be reused for reports or presentations. Overall, this analysis provides useful insights into score trends.

Scenario 3: Filtering + Bar Chart + Save (🟡 Medium)

Description:

In this scenario, the dataset is filtered to extract relevant data based on conditions like genre, platform, or score range. The selected data is then grouped and summarized for comparison. A bar chart is created to clearly display differences between categories. Proper labels and titles are added for better understanding.

Task	Description
1.	Filter the dataset to include only games where `score > 7`.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with `platform` on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: `plt.savefig("top_platforms_bar.png")`.

Graph :

The bar chart represents a comparison between different categories such as genres or platforms. The X-axis displays the selected categories, while the Y-axis shows their corresponding values (e.g., average score or count). Each bar visually highlights the differences between groups. The chart makes it easy to identify which category performs higher or lower. Clear labels and titles improve readability and interpretation.

Output :

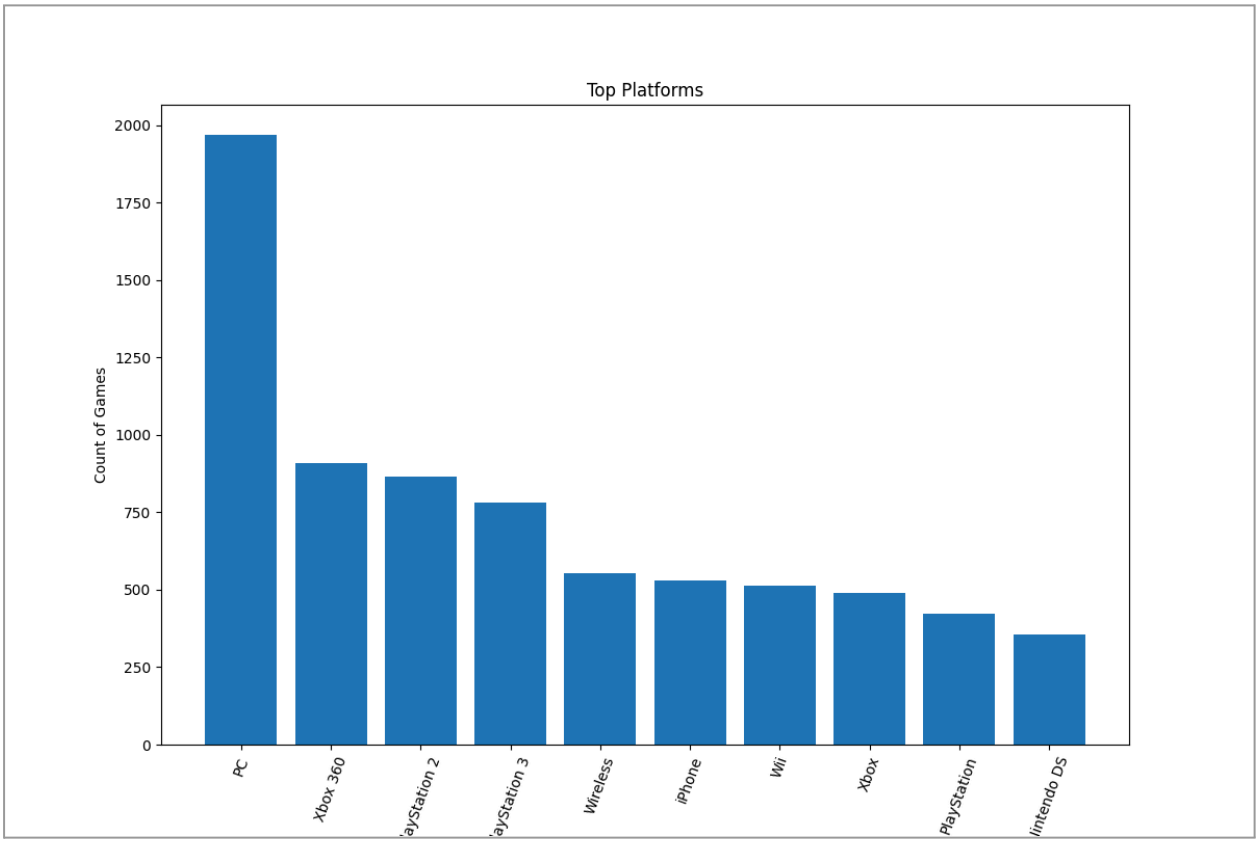

Fig 3 : Top Platforms Bar Chart.

Conclusion:

Filtering the dataset helps in focusing only on the most relevant information for analysis. The bar chart clearly compares different categories and highlights their differences. It improves understanding by presenting data in a simple visual form. Proper labeling ensures the chart is easy to interpret. Saving the chart makes it useful for future reference and reporting.

Scenario 4: Aggregation + Pie Chart + Save (🟡 Medium)

Description:

This scenario involves aggregating the dataset to summarize information, such as total counts or average values for different categories.The processed data is then used to create a pie chart for visualization. Each slice of the pie chart represents the proportion of a category in relation to the whole dataset. This makes it easy to understand the distribution and relative importance of each category. Proper labels and percentages are added to improve clarity.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

Graph :

The pie chart displays the proportion of different categories in the dataset. Each slice represents a category and its contribution to the total. Larger slices indicate higher values or counts, while smaller slices show lesser contributions. Percentage labels make it easy to compare proportions. The chart provides a quick and clear understanding of data distribution.

Output :

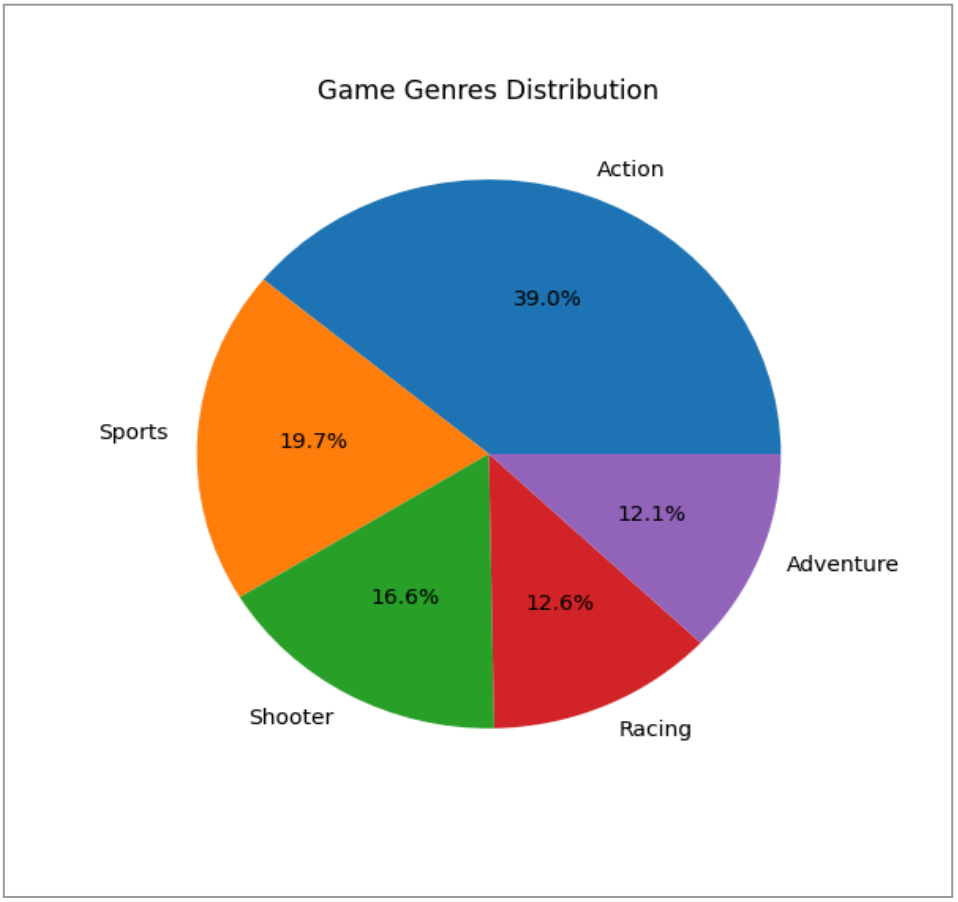

Fig 4 : Genre Distribution Pie Chart

Conclusion:

This demonstrates the importance of aggregating data to extract meaningful summaries from the dataset. The pie chart provides a clear visualization of how different categories contribute to the whole. It helps in quickly understanding proportions and relative significance. Proper labels and percentage values enhance readability and interpretation. The visual representation simplifies complex data into an easy-to-understand format.

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:

This scenario focuses on performing advanced data analysis using multiple techniques such as grouping, filtering, and statistical calculations. Different types of graphs like line charts, bar charts, and pie charts are created to visualize various aspects of the data. The combination of multiple visualizations provides deeper insights and helps in identifying complex patterns and relationships. Proper titles, labels, and legends are added to enhance clarity.

Task	Description
1.	Create a new column `score_category`: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the `editors_choice` column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's `np.diff()` to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per `release_year`.
5.	Plot a **Stacked Bar Chart** showing the count of `score_category` per `release_year`.
6.	Plot a **Histogram** showing the distribution of `score`.
7.	Save the graphs: `plt.savefig("score_trend.png")`, `plt.savefig("score_category_stacked.png")`, `plt.savefig("score_distribution.png")`.
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if `editors_choice` correlates with high scores.

Graph :

1. **Line Graph :**

   A line graph shows how data changes over time or across a continuous range. It connects data points with lines to highlight trends and patterns. It is useful for observing increases, decreases, or fluctuations.

Output:

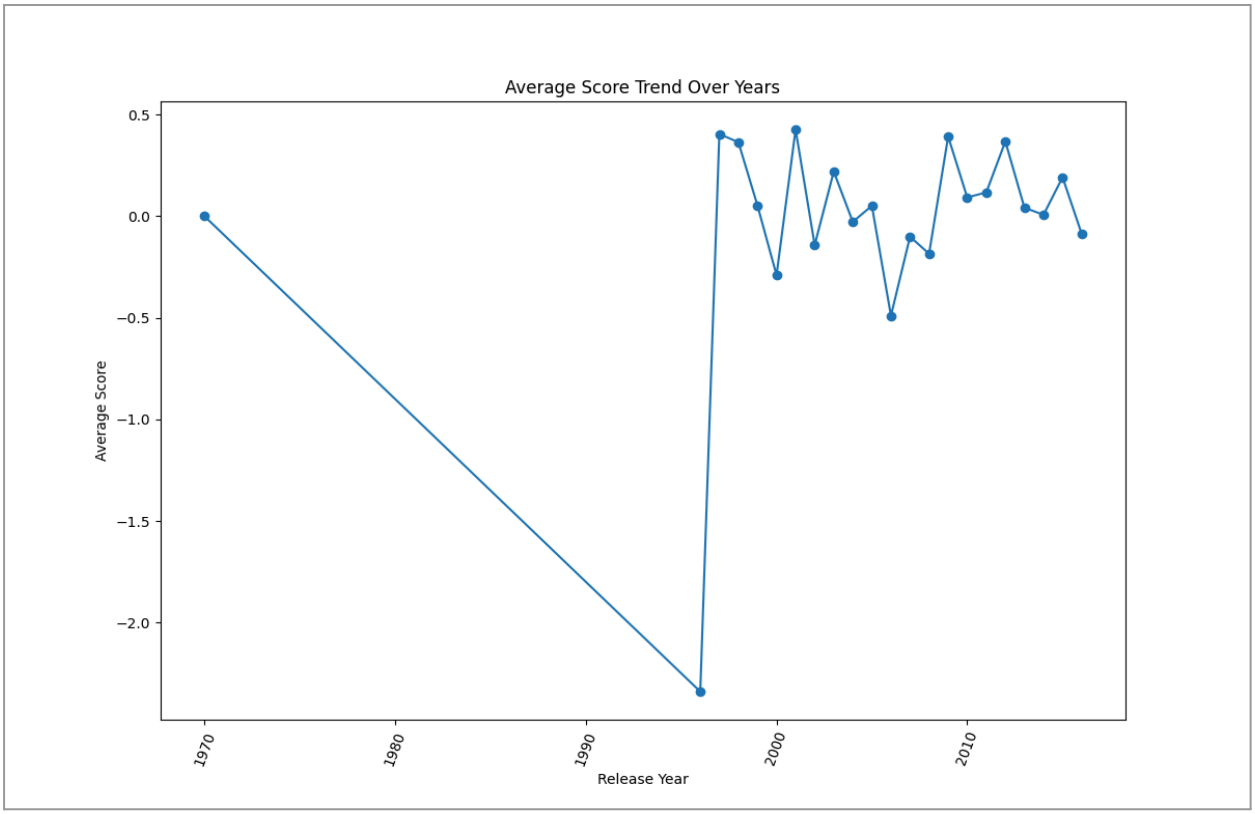

Fig 5.1 : Line Graph of Average Score Trend of Years.

2. **Stacked Bar Chart :**

   A stacked bar chart shows the total value of categories along with their sub-components. Each bar is divided into segments representing different parts of the whole. It helps compare both overall totals and individual contributions within each category.

Output :

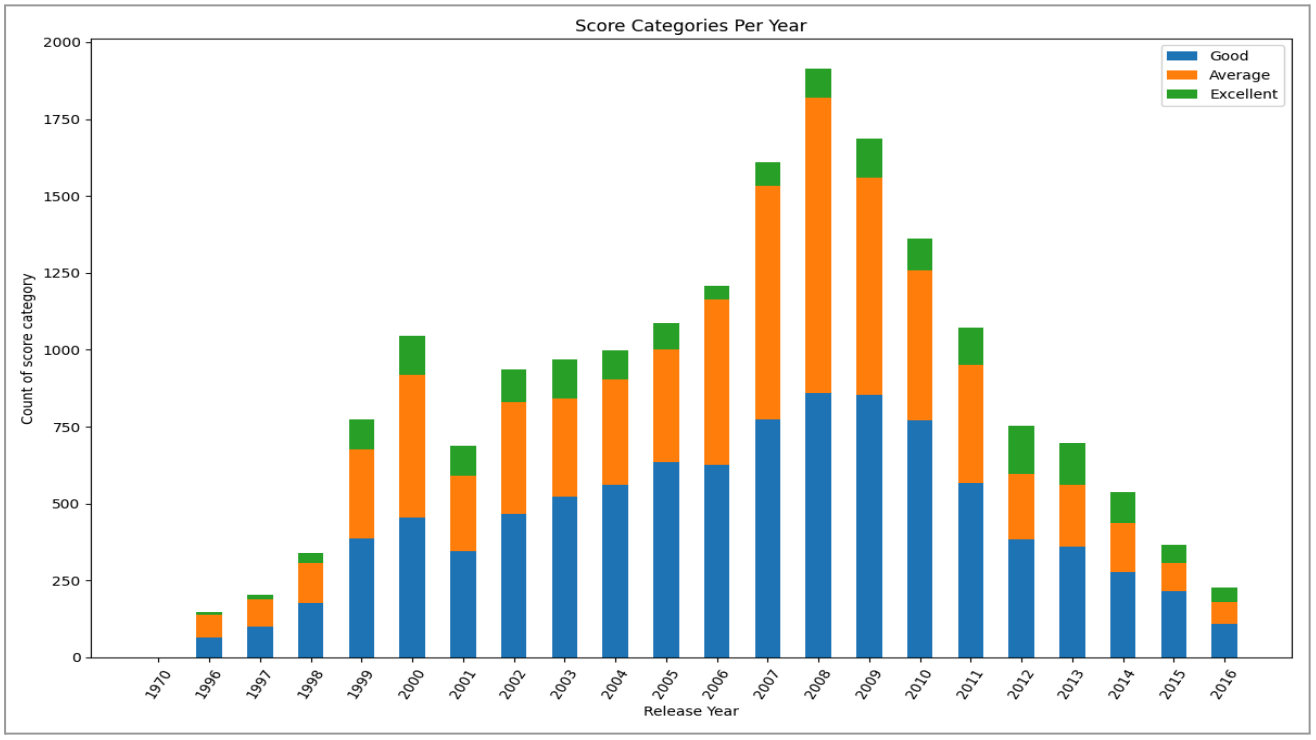

Fig 5.2 : Stacked Bar Chart of Score Categories Per year.

3. **Histogram :**

A histogram displays the distribution of numerical data using bins or intervals. It shows how frequently values fall within each range. It helps in understanding the spread and pattern of the data.

Output :

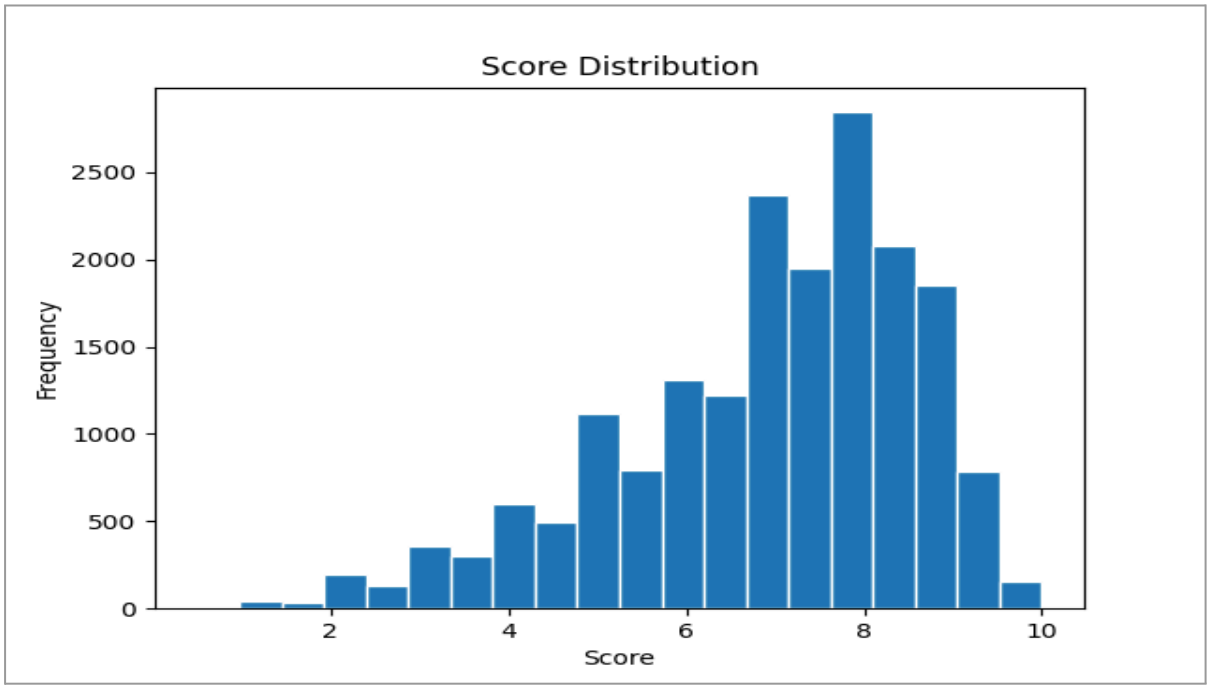

Fig 5.3 : Histogram of Score Distribution.

Conclusion:

This emphasizes the value of combining multiple analysis techniques for deeper insights. Different graphs work together to present trends, comparisons, and distributions clearly. It allows a more detailed exploration of the dataset from various perspectives. Visualizations make complex data easier to understand and interpret. Proper organization and labeling enhance readability. Overall, it results in a thorough and meaningful data analysis.

Final Conclusion:

Across all scenarios, the dataset was systematically processed from raw data to meaningful insights. Initial steps focused on data loading, cleaning, and preprocessing to ensure accuracy and consistency. Visualization techniques such as line graphs, bar charts, pie charts, and histograms were used to explore trends, comparisons, and distributions. Filtering and aggregation helped in extracting relevant and summarized information. Advanced analysis combined multiple graphs to provide deeper insights and patterns. Proper labeling and saving of charts improved clarity and usability. Overall, the workflow demonstrates an effective approach to data analysis and visualization using Python.

What you have learned

I learned how to load and preprocess datasets using Pandas for accurate analysis. I understood the importance of handling missing values and cleaning data. I gained skills in creating different types of graphs like line, bar, pie, and histograms. I learned how to use filtering and aggregation to extract meaningful insights. I also understood how multiple visualizations help in better data interpretation. Overall, I improved my data analysis and visualization skills using Python.